Estrutura do Projeto — Sistema de Reservas Mútua-PI

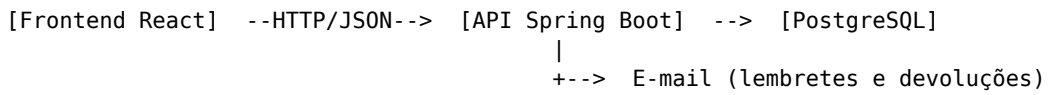
Gestão de ambientes compartilhados e equipamentos do Crea Tec HUB · Proposta de arquitetura e organização do MVP · ICEV, outubro de 2026

1. Contexto

A Mútua-PI está concluindo a reforma da sua sede, que passará a oferecer ambientes compartilhados (coworking, sala de estudos, salas multiuso, sala de reunião e auditório) e equipamentos do Crea Tec HUB (drones, pacômetros, medidores de pressão sonora, câmeras termográficas, entre outros). Hoje não há ferramenta para controlar reservas, retiradas e devoluções, o que gera risco de conflitos de horário, falta de histórico e baixa rastreabilidade dos bens. Este documento propõe a estrutura técnica do MVP.

2. Arquitetura geral

Aplicação web responsiva: o profissional usa principalmente pelo celular e a equipe administrativa pelo computador. O sistema é dividido em três partes.



Camada	Tecnologia sugerida
Back-end (API)	Java 21 + Spring Boot (Web, Data JPA, Security, Validation)
Front-end	React + Vite + TypeScript
Banco de dados	PostgreSQL
Autenticação	JWT com perfis PROFISSIONAL e ADMIN
Infraestrutura	Docker Compose (sobe API, front e banco com um comando)
Integração contínua	GitHub Actions (build e testes a cada Pull Request)

Se o grupo dominar outra stack, a organização proposta continua válida.

3. Estrutura do repositório (monorepo)

```
mutua-reservas/
├── backend/
│   └── src/main/java/br/icev/mutua/
│       ├── auth/          # login, JWT, perfis
│       ├── usuario/       # cadastro de profissionais e admins
│       ├── recurso/       # ambientes e equipamentos
│       ├── regra/         # regras de uso por recurso
│       ├── reserva/       # reservas e prevenção de conflitos
│       ├── emprestimo/    # retirada e devolução de equipamentos
│       ├── notificacao/   # e-mails e lembretes agendados
│       ├── relatorio/     # indicadores e histórico
│       └── common/        # exceções, configuração, utilitários
├── frontend/
│   └── src/
│       ├── pages/profissional/   # disponibilidade, minhas reservas
│       ├── pages/admin/          # painel, cadastros, retiradas
│       ├── components/           # calendário, cards, formulários
│       ├── services/             # chamadas à API
│       └── hooks/
├── docs/                # requisitos, diagramas, TAP, atas
├── docker-compose.yml
├── .github/workflows/   # CI
└── README.md
```

Cada módulo do back-end tem suas próprias camadas (controller, service, repository, dto, entity). Organizar por funcionalidade, e não por camada, permite que cada integrante trabalhe na sua pasta com poucos conflitos de merge.

4. Modelo de dados

Entidade	Campos principais
Usuario	nome, e-mail, CPF, nº de registro no Crea, perfil (PROFISSIONAL / ADMIN)
Recurso	nome, tipo (COWORKING, SALA, AUDITORIO, EQUIPAMENTO), capacidade, nº de patrimônio, ativo
RegraUso	recurso, duração máxima, antecedência mínima e máxima, horário de funcionamento, exige aprovação
Reserva	usuário, recurso, início, fim, status
Emprestimo	reserva, data de retirada, condição na retirada, data de devolução, condição na devolução, admin responsável, observações/fotos
Notificacao	usuário, reserva, tipo, data de envio

Decisões importantes

- Recurso único com campo tipo: ambientes e equipamentos ficam na mesma entidade. Um novo espaço ou equipamento vira apenas um cadastro, atendendo ao requisito de expansão futura sem alterar código.
- Ciclo de vida da reserva: PENDENTE → CONFIRMADA → EM_USO → CONCLUIDA, além de CANCELADA e NAO_COMPARECEU.
- Prevenção de conflitos em duas camadas: validação no service e uma exclusion constraint do PostgreSQL, que impede no próprio banco duas reservas sobrepostas para o mesmo recurso, mesmo com requisições simultâneas.

5. Escopo: MVP e versões futuras

MVP (entrega do semestre)	Fase 2 (roadmap)
Login e cadastro de profissional e admin	Lembretes por e-mail ou WhatsApp
Cadastro de ambientes e equipamentos	Regras avançadas e fluxo de aprovação
Consulta de disponibilidade em calendário	Relatórios com gráficos (ocupação, uso por equipamento)
Reserva com bloqueio automático de conflito	Fotos da condição do equipamento
Cancelamento e alteração de reserva	Lista de espera
Registro de retirada e devolução com condição	Penalidade por não comparecimento
Painel admin com as reservas do dia	Aplicativo mobile nativo, se necessário
Histórico por usuário e por recurso	

6. Divisão do grupo (exemplo com 5 integrantes)

Função	Responsabilidades
Back-end 1	Módulos auth, usuario e recurso
Back-end 2	Módulos reserva, regra e emprestimo (regra de conflito)
Front-end 1	Telas do profissional: disponibilidade, reserva, minhas reservas
Front-end 2	Painel administrativo: cadastros, retiradas/devoluções, histórico
PO / QA / Docs	Levantamento com a Mútua-PI, pasta docs, testes, README e apresentação

Com menos integrantes, as duas funções de front-end podem ser unidas, ou o papel de PO/QA pode ser acumulado por um dos desenvolvedores.

7. Fluxo de trabalho no GitHub

- main: versão estável, protegida (só recebe código via Pull Request aprovado).
- develop: integração das funcionalidades.
- feature/nome-da-tarefa: uma branch por tarefa, aberta a partir da develop.
- Cada tarefa vira uma Issue atribuída a um integrante e acompanhada no quadro do GitHub Projects.
- Commits padronizados (feat:, fix:, docs:) e ninguém faz merge do próprio Pull Request.

```
git checkout develop && git pull origin develop
git checkout -b feature/tela-reserva
git add . && git commit -m "feat: cria tela de reserva"
git push origin feature/tela-reserva   # depois abrir o Pull Request para develop
```

8. Cronograma em sprints (2 semanas cada)

Sprint	Entregas
Sprint 1	Repositório, Docker, banco de dados, login e cadastro de recursos

Sprint	Entregas
Sprint 2	Reservas com prevenção de conflitos e calendário no front-end
Sprint 3	Retirada e devolução, histórico e painel administrativo
Sprint 4	Testes, ajustes da validação com a Mútua-PI, deploy e apresentação